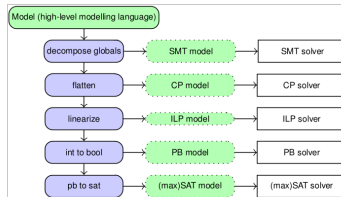


L07: Solving technologies and encodings, part 2



Prof. Tias Guns and Dr. Hendrik ‘Henk’ Bierlee

KU LEUVEN

Partly based on slides from Pierre Flener, Ambros Gleixner and Jakob Nordström

Recap, goal of the lecture

Part 2 of solving technologies and encodings.

Part 1 covered SMT, decomposing global constraints, and CP solving (search + propagation).

This lecture: flatten nested expressions; ILP (+ linearisation); PB; $\text{int} \rightarrow \text{bool}$ encodings; SAT/CDCL; $\text{PB} \rightarrow \text{SAT}$; choosing a technology.

Recap, goal of the lecture

Part 2 of solving technologies and encodings.

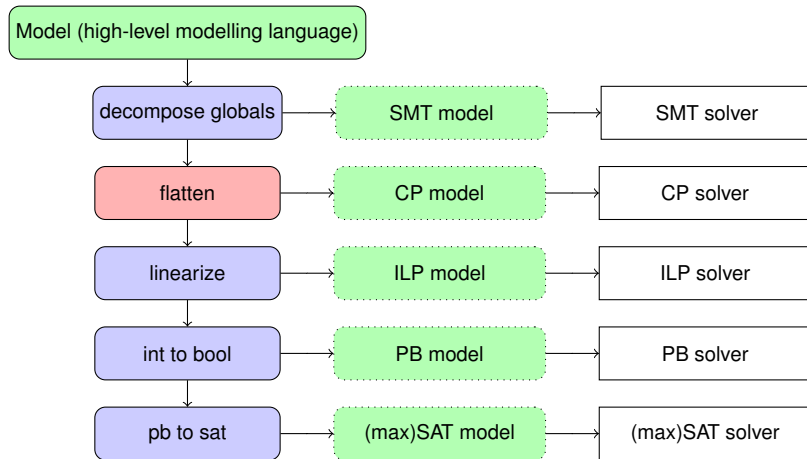
Part 1 covered SMT, decomposing global constraints, and CP solving (search + propagation).

This lecture: flatten nested expressions; ILP (+ linearisation); PB; $\text{int} \rightarrow \text{bool}$ encodings; SAT/CDCL; $\text{PB} \rightarrow \text{SAT}$; choosing a technology.

Still the same goals:

- ▶ to understand advantages and limitations of each technology;
- ▶ to help you choose a technology for a particular model;
- ▶ to help you encode and adapt a model to a particular technology.

Overview



From CP Language to CP Solver: primitive constraints

A CP solver accepts as input a number of *primitive constraints*, that each will have a propagator that can do **inference**.

Example primitive constraints, note their arguments are *variables*:

$$a \vee b$$

$$x \neq y$$

$$x + y \geq 2$$

$$x + 2 \cdot y \neq 3$$

$$\text{ALLDIFFERENT}(x, y, z)$$

$$\text{COUNTEQ}([x, y, z], 0, r)$$

From CP Language to CP Solver: primitive constraints

A CP solver accepts as input a number of *primitive constraints*, that each will have a propagator that can do **inference**.

Also primitive for many CP solvers: one level of reification (Boolean decision variable on left-hand-side):

$$b = (x = 0)$$

$$b \rightarrow x \neq 3$$

$$b \rightarrow (y + z \leq 3)$$

$$b \rightarrow (5 \cdot y - 2 \cdot z \leq 3)$$

From CP Language to CP Solver: non-primitive constraints

Example **nested** constraints, not primitive because their arguments contain *sub-expressions*:

$$\neg(a \vee b)$$

$$\neg(x > y)$$

$$a \rightarrow (b \vee c)$$

$$a \vee (x + y \geq 2)$$

$$x + (y \cdot z) \geq 0$$

$$(a \vee b) = (x = 0)$$

$$x + 3 \neq \text{COUNT}([y, z, q], 0)$$

$$\text{ALLDIFFERENT}(x + 2, y - 1, z)$$

$$\text{COUNTEQ}([x + 2, y - 1, z], 0, r + 2 \cdot s)$$

From CP Language to CP Solver: negation

Assume all unsupported global constraints have been decomposed already.

Eliminating negation

Negation $\neg(E)$ is not a primitive constraint of CP solvers, because it can be *eliminated*, by applying a set of rewrite rules over Boolean expressions:

- ▶ $\neg b$ with b a Boolean variable is the only allowed negated expression. Recall that b and $\neg b$ are called the *literals* of Boolean variable b .
- ▶ $\neg(x \text{ CMP } y)$, with CMP a comparison relation, has standard mathematical negation rules: $= \Leftrightarrow \neq$, $> \Leftrightarrow \leq$, $< \Leftrightarrow \geq$.
- ▶ $\neg(\neg(E)) \Leftrightarrow E$.

From CP Language to CP Solver: negation

Assume all unsupported global constraints have been decomposed already.

Eliminating negation

Negation $\neg(E)$ is not a primitive constraint of CP solvers, because it can be *eliminated*, by applying a set of rewrite rules over Boolean expressions:

- ▶ $\neg b$ with b a Boolean variable is the only allowed negated expression. Recall that b and $\neg b$ are called the *literals* of Boolean variable b .
- ▶ $\neg(x \text{ CMP } y)$, with CMP a comparison relation, has standard mathematical negation rules: $= \Leftrightarrow \neq$, $> \Leftrightarrow \leq$, $< \Leftrightarrow \geq$.
- ▶ $\neg(\neg(E)) \Leftrightarrow E$.

The next two can now be applied recursively to eliminate negated expressions.

- ▶ $\neg(E_1 \wedge E_2)$, by DeMorgan's law is equivalent to $(\neg E_1) \vee (\neg E_2)$.
- ▶ $\neg(E_1 \vee E_2)$, also by DeMorgan's law equivalent to $(\neg E_1) \wedge (\neg E_2)$. Likewise $\neg(E_1 \rightarrow E_2)$ is equivalent to $E_1 \wedge (\neg E_2)$ because $(E_1 \rightarrow E_2) \Leftrightarrow ((\neg E_1) \vee E_2)$.

From CP Language to CP Solver: flattening

Flattening is the process of replacing subexpressions by fresh auxiliary variables, with an additional constraint that enforces the variable to equal the subexpression.

Numeric subexpressions

Create a new auxiliary decision variable with the same bounds as the subexpression and enforce equality:

Example: $x + (y \cdot z) \geq 0 \Leftrightarrow x + r \geq 0 \wedge (r = y \cdot z)$

From CP Language to CP Solver: flattening

Flattening is the process of replacing subexpressions by fresh auxiliary variables, with an additional constraint that enforces the variable to equal the subexpression.

Numeric subexpressions

Create a new auxiliary decision variable with the same bounds as the subexpression and enforce equality:

Example: $x + (y \cdot z) \geq 0 \Leftrightarrow x + r \geq 0 \wedge (r = y \cdot z)$

Boolean subexpressions

Example: $x + \llbracket y > z \rrbracket = 3 \Leftrightarrow x + e = 3 \wedge (e \leftrightarrow y > z)$.

Special cases: $x + \llbracket y > z \rrbracket \geq 3$ and $a \vee (y > z)$ only care about the truthness of $(y > z)$, so $e \rightarrow y > z$ is sufficient.

Likewise, $x + \llbracket y > z \rrbracket \leq 3$ only cares about falseness, so $e \leftarrow y > z$ is sufficient.

From CP Language to CP Solver: flattening

Flattening is the process of replacing subexpressions by fresh auxiliary variables, with an additional constraint that enforces the variable to equal the subexpression.

Numeric subexpressions

Create a new auxiliary decision variable with the same bounds as the subexpression and enforce equality:

Example: $x + (y \cdot z) \geq 0 \Leftrightarrow x + r \geq 0 \wedge (r = y \cdot z)$

Boolean subexpressions

Example: $x + \llbracket y > z \rrbracket = 3 \Leftrightarrow x + e = 3 \wedge (e \leftrightarrow y > z)$.

Special cases: $x + \llbracket y > z \rrbracket \geq 3$ and $a \vee (y > z)$ only care about the truthness of $(y > z)$, so $e \rightarrow y > z$ is sufficient.

Likewise, $x + \llbracket y > z \rrbracket \leq 3$ only cares about falseness, so $e \leftarrow y > z$ is sufficient.

Common subexpression elimination: if a subexpression already has an auxiliary variable created for it, reuse that.

From CP Language to CP Solver: simplification and normalisation

If we can **avoid creating auxiliary variables**, we should: more compact formulation, and the solver can reason more directly on the original variables.

We can do this by **simplifying** and **normalizing** expressions, before flattening their subexpressions.

- ▶ Normalizing linear sums: $x + y + 2 = z + 3 \Leftrightarrow x + y - z = 1$.
- ▶ Merge nested n-ary operators, e.g. $a \vee \neg(b \wedge c) \Leftrightarrow a \vee (\neg b \vee \neg c)$ (by negation) $\Leftrightarrow a \vee \neg b \vee \neg c$ (by merging into a single disjunction)
also for weighted sums: $x + 2 \cdot (y - 3 \cdot z) \geq 0 \Leftrightarrow x + 2 \cdot y - 6 \cdot z \geq 0$.

All of these rules also hold if a, b, c, x, y, z are not variables but other expressions.

From CP Language to CP Solver: simplification and normalisation

We can do additional optimisations to avoid nested Boolean expressions:

- Can rewrite (nested) implications into disjunctions, because

$$a \rightarrow b \Leftrightarrow (\neg a) \vee b.$$

$$\text{So } a \vee (b \rightarrow c) \Leftrightarrow a \vee (\neg b \vee c) \Leftrightarrow a \vee \neg b \vee c.$$

$$\text{Also } a \rightarrow (b \rightarrow c) \Leftrightarrow a \rightarrow (\neg b \vee c) \Leftrightarrow \neg a \vee \neg b \vee c$$

$$\text{and } (a \wedge b) \rightarrow c \Leftrightarrow \neg a \vee \neg b \vee c.$$

From CP Language to CP Solver: simplification and normalisation

We can do additional optimisations to avoid nested Boolean expressions:

- ▶ Can rewrite (nested) implications into disjunctions, because

$$a \rightarrow b \Leftrightarrow (\neg a) \vee b.$$

$$\text{So } a \vee (b \rightarrow c) \Leftrightarrow a \vee (\neg b \vee c) \Leftrightarrow a \vee \neg b \vee c.$$

$$\text{Also } a \rightarrow (b \rightarrow c) \Leftrightarrow a \rightarrow (\neg b \vee c) \Leftrightarrow \neg a \vee \neg b \vee c$$

$$\text{and } (a \wedge b) \rightarrow c \Leftrightarrow \neg a \vee \neg b \vee c.$$

- ▶ Can use distributivity to rewrite nested disjunction-with-conjunctions, e.g.

$$a \vee (b \wedge c) \Leftrightarrow (a \vee b) \wedge (a \vee c)$$

so no auxiliary variable for the nested $(b \wedge c)$ needed.

- ▶ Likewise, 2 special cases for implication: $a \rightarrow (b \wedge c) \Leftrightarrow (a \rightarrow b) \wedge (a \rightarrow c)$
as well as $(a \vee b) \rightarrow c \Leftrightarrow (a \rightarrow c) \wedge (b \rightarrow c).$

From CP Language to CP Solver: simplification and normalisation

We can do additional optimisations to avoid nested Boolean expressions:

- Can rewrite (nested) implications into disjunctions, because

$$a \rightarrow b \Leftrightarrow (\neg a) \vee b.$$

$$\text{So } a \vee (b \rightarrow c) \Leftrightarrow a \vee (\neg b \vee c) \Leftrightarrow a \vee \neg b \vee c.$$

$$\text{Also } a \rightarrow (b \rightarrow c) \Leftrightarrow a \rightarrow (\neg b \vee c) \Leftrightarrow \neg a \vee \neg b \vee c$$

$$\text{and } (a \wedge b) \rightarrow c \Leftrightarrow \neg a \vee \neg b \vee c.$$

- Can use distributivity to rewrite nested disjunction-with-conjunctions, e.g.

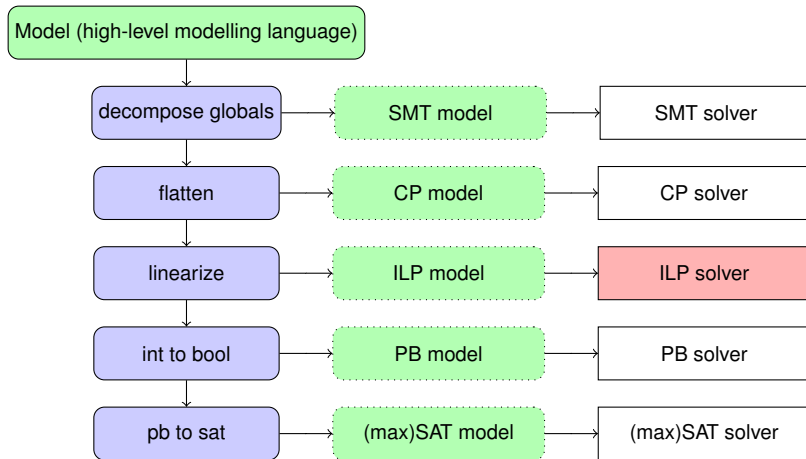
$$a \vee (b \wedge c) \Leftrightarrow (a \vee b) \wedge (a \vee c)$$

so no auxiliary variable for the nested $(b \wedge c)$ needed.

- Likewise, 2 special cases for implication: $a \rightarrow (b \wedge c) \Leftrightarrow (a \rightarrow b) \wedge (a \rightarrow c)$
as well as $(a \vee b) \rightarrow c \Leftrightarrow (a \rightarrow c) \wedge (b \rightarrow c).$

Note, we keep the outer-most implications because $b \rightarrow (y + z \leq 3)$ and $b \rightarrow \text{GLOBALCONSTRAINT}()$ are valid primitive constraints.

Overview



Mixed Integer Linear Programming (M)ILP

Mixed = both discrete and continuous decision variables

MIP primitive constraints and objective:

$$\text{minimize} \quad \sum_j c_j x_j$$

$$\text{subject to} \quad \sum_j a_{ij} x_j \geq b_i \quad \text{for } i = 1, \dots, m$$

$$x_j \in \{0, 1\} \text{ or } \mathbb{Z} \text{ or } \mathbb{R} \quad \cap \quad [\ell_j, u_j] \quad (\text{possibly } \pm\infty)$$

Noteworthy characteristics:

- ▶ Boolean, integer, as well as real-valued decision variables x_j .
- ▶ Coefficients a_{ij} , b_i can be fractional
- ▶ Decision variables can be unbounded

Mixed Integer Linear Programming (M)ILP: inequalities

Only $\sum_j a_{ij}x_j \geq b_i$ as primitive constraint?

Many constraints can be rewritten as inequalities as-is:

- ▶ $\sum_j a_{ij}x_j \leq b_i \Leftrightarrow \sum_j -a_{ij}x_j \geq -b_i$
- ▶ $\sum_j a_{ij}x_j = b_i \Leftrightarrow \sum_j a_{ij}x_j \geq b_i \wedge \sum_j a_{ij}x_j \leq b_i$
(note that this only guarantees bounds consistency on each inequality)
- ▶ If b_i is integer and $\sum_j a_{ij}x_j$ is integer (e.g. because all x_j and a_{ij} are integer):
 $\sum_j a_{ij}x_j > b_i \Leftrightarrow \sum_j a_{ij}x_j \geq b_i + 1$

Mixed Integer Linear Programming (M)ILP: inequalities

Only $\sum_j a_{ij}x_j \geq b_i$ as primitive constraint?

Many constraints can be rewritten as inequalities as-is:

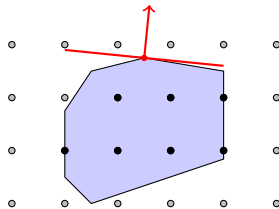
- ▶ $\sum_j a_{ij}x_j \leq b_i \Leftrightarrow \sum_j -a_{ij}x_j \geq -b_i$
- ▶ $\sum_j a_{ij}x_j = b_i \Leftrightarrow \sum_j a_{ij}x_j \geq b_i \wedge \sum_j a_{ij}x_j \leq b_i$
(note that this only guarantees bounds consistency on each inequality)
- ▶ If b_i is integer and $\sum_j a_{ij}x_j$ is integer (e.g. because all x_j and a_{ij} are integer):
 $\sum_j a_{ij}x_j > b_i \Leftrightarrow \sum_j a_{ij}x_j \geq b_i + 1$
- ▶ $x_1 \vee x_2 \vee x_3 \Leftrightarrow x_1 + x_2 + x_3 \geq 1$
- ▶ $x_1 \vee \neg x_2 \vee x_3 \Leftrightarrow x_1 + (1 - x_2) + x_3 \geq 1 \Leftrightarrow x_1 - x_2 + x_3 \geq 0$

Others less straightforward, see the *linearize* transformation later.

Mixed Integer Linear Programming (M)ILP: inference

The **LP relaxation** of a MILP removes the integrality requirements:
 $\{\ell, \dots, u\} \rightarrow [\ell, u]$ especially $\{0, 1\} \rightarrow [0, 1]$.

$$\begin{aligned} &\text{minimize} && \sum_j c_j x_j \\ &\text{subject to} && \sum_j a_{ij} x_j \geq b_i \quad \text{for } i = 1, \dots, m \\ &&& x_j \in \mathbb{R} \cap [\ell_j, u_j] \quad (\text{possibly } \pm\infty) \end{aligned}$$



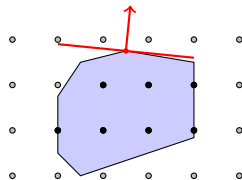
Useful properties of the LP relaxation:

- ▶ The constraint set $\sum_j a_{ij} x_j \geq b_i$ forms a convex polytope, and the optimal solution can be found in **polynomial time** (so efficiently)
- ▶ Its solution is **informative** for the (M)ILP solution

Mixed Integer Linear Programming (M)ILP: inference

LP relaxation:

$$\begin{aligned} &\text{minimize} && \sum_j c_j x_j \\ &\text{subject to} && \sum_j a_{ij} x_j \geq b_i \quad \text{for } i = 1, \dots, m \\ &&& x_j \in \mathbb{R} \cap [\ell_j, u_j] \quad (\text{possibly } \pm\infty) \end{aligned}$$



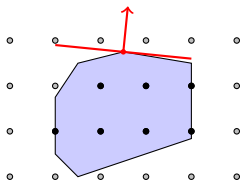
Inference with the LP relaxation:

- ▶ If the LP relaxation has **no solution**, the MIP problem has no solution either
- ▶ If the **optimal solution** to the LP relaxation is integral, then it is an optimal solution to the ILP problem (similarly for a mixed problem)

What if it has a solution that is not fully integral?

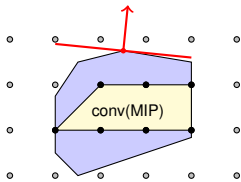
Mixed Integer Linear Programming (M)ILP: inference 2

What if the LP relaxation solution is not fully integral?



Mixed Integer Linear Programming (M)ILP: inference 2

What if the LP relaxation solution is not fully integral?

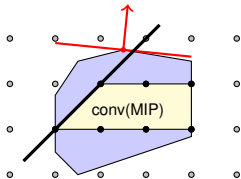


Observe: there exists a set of constraints such that an LP relaxed solution is the MILP solution: the convex hull of the integer feasible solutions.

- ▶ But the linear description is not known...
- ▶ and it has a large number of constraints.

Mixed Integer Linear Programming (M)ILP: inference 2

What if the LP relaxation solution is not fully integral?



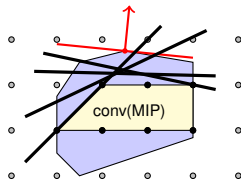
Inference method 2: cutting planes

Inferring additional constraints (cuts) that cut away the LP solution and bring us close to the MILP solution (without cutting integral solutions away).

Mixed Integer Linear Programming (M)ILP: inference 2

Many cutting plane generation algorithms exist:

- ▶ Gomory cuts (for pure integer programs, guaranteed to converge)
- ▶ Mixed Integer Rounding (MIR) cuts
- ▶ Cover cuts
- ▶ Lift-and-project cuts
- ▶ ...



Cutting plane generation is a powerful inference method, but in practice:

- ▶ Slow to converge
- ▶ Smart cut selection as well as aging is important
- ▶ Best combined with branch-and-bound: branch-and-cut

Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

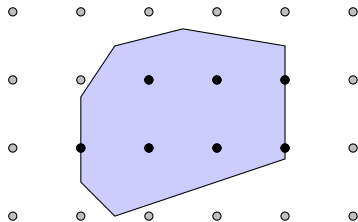
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

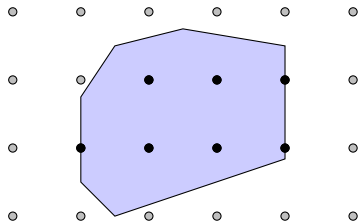
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

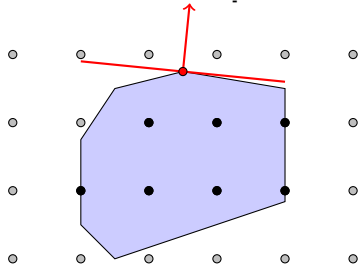
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



(For illustration purposes, no cutting planes are generated)

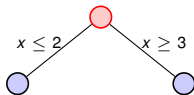
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

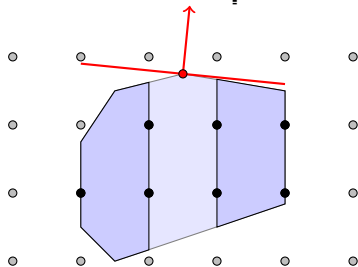
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



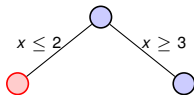
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

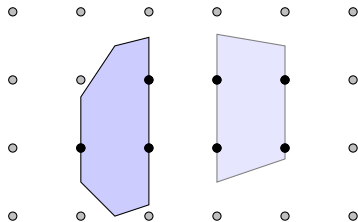
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



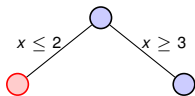
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

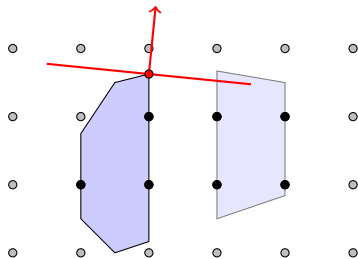
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



(For illustration purposes, no cutting planes are generated)

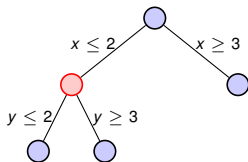
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

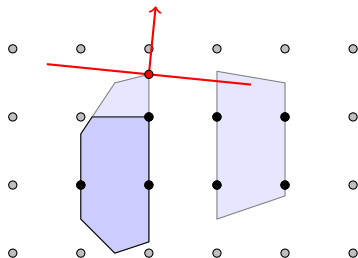
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



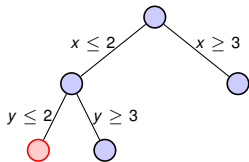
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

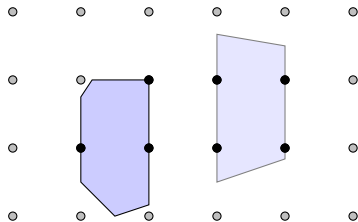
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



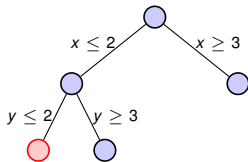
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

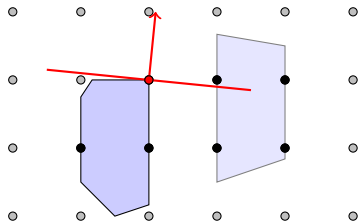
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



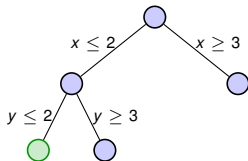
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

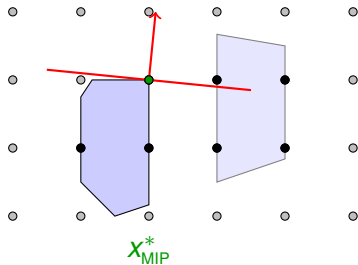
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



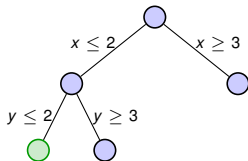
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

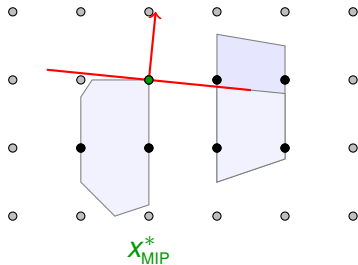
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



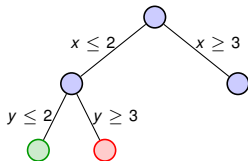
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

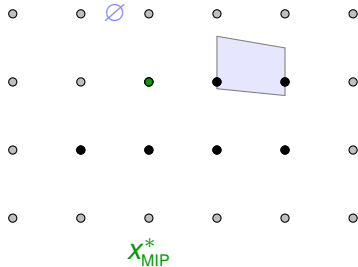
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



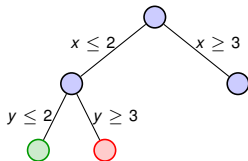
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

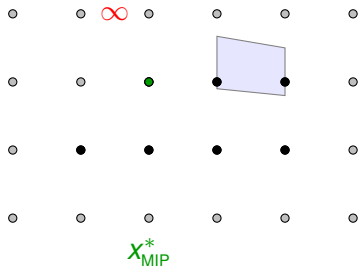
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



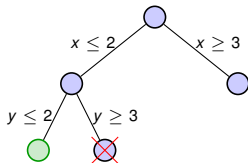
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

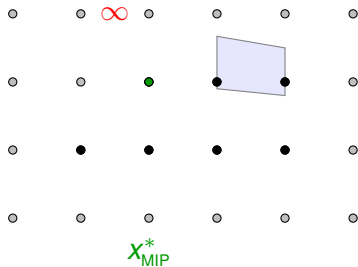
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



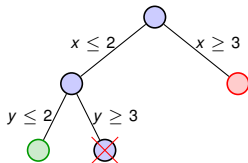
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

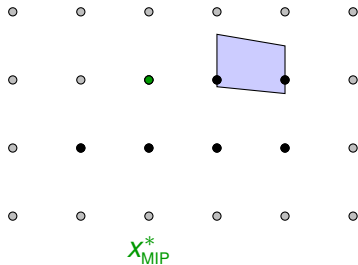
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



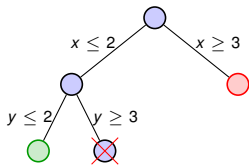
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

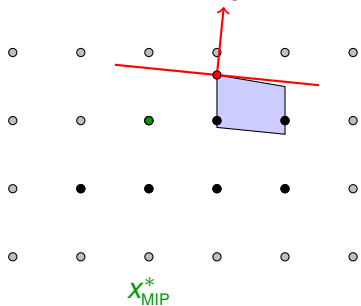
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



(For illustration purposes, no cutting planes are generated)

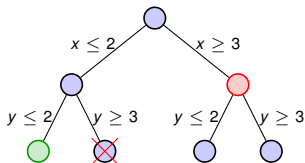
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

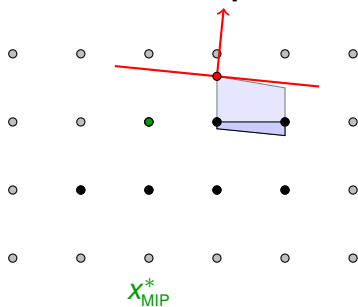
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



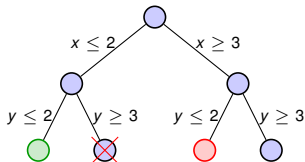
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

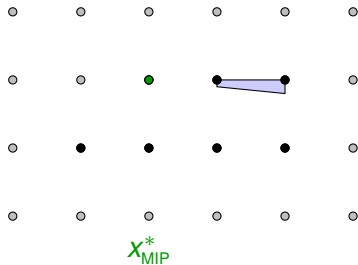
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



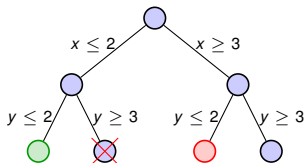
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

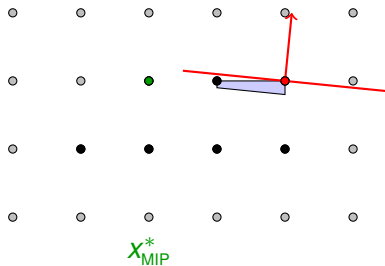
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



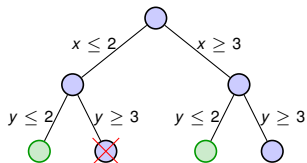
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

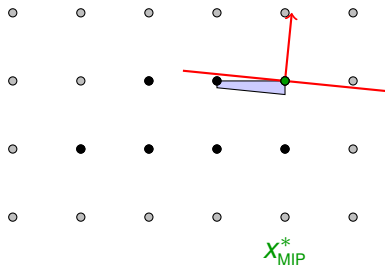
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



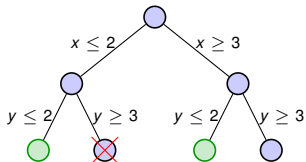
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

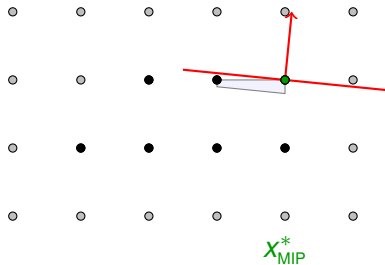
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



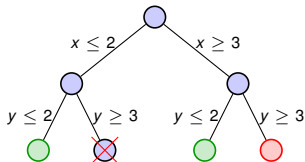
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

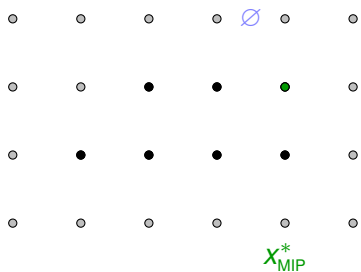
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



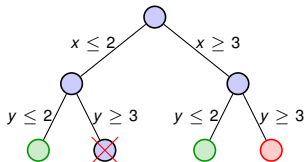
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

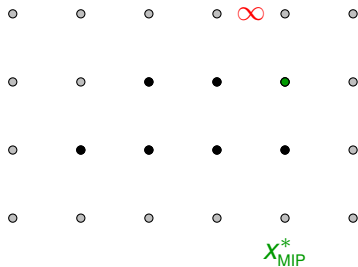
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space



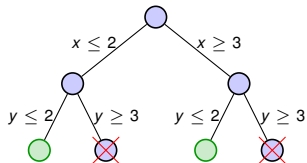
Branch-and-cut

Limited cutting plane generation at root and nodes of a branch-and-bound tree.

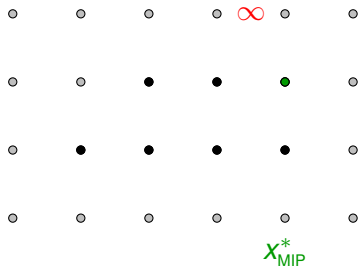
LP-based branch-and-bound:

Systematic reduction of $UB - LB$ by divide-and-conquer.

Branch-and-bound tree



Solution space

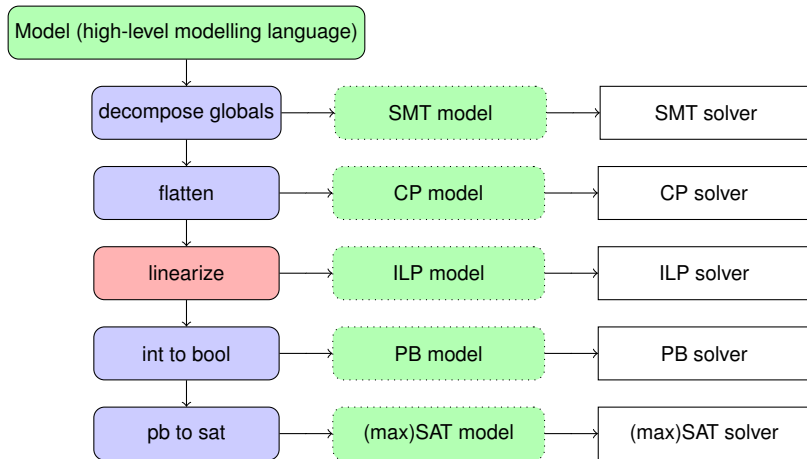


Efficient MILP solvers

Many additional techniques:

- ▶ incremental LP computation: each LP call is *warmstarted* from previous LP solution
- ▶ pre-solving: more effort in the root node
- ▶ primal heuristics (e.g. local search)
- ▶ branching heuristics and node selection
- ▶ cut generation, selection, and *management*
- ▶ symmetry detection and handling
- ▶ parallelization
- ▶ ...

Overview



Linearisation of flat models

Assume all unsupported global constraints are decomposed, and all constraints are flattened.

We've covered how $\sum_j a_{ij}x_j \leq b_i$, $\sum_j a_{ij}x_j = b_i$, and $\sum_j a_{ij}x_j > b_i$ can be rewritten to $\sum_j a_{ij}x_j \geq b$ form. Same for disjunctions $x_1 \vee \neg x_2 \vee x_3$.

What about **reification** $d \rightarrow \text{Expr}$ (with d a Boolean decision variable)?

Linearisation of implications

Let's recap the truth table of $d \rightarrow e$:

d	e	$d \rightarrow e$
T	T	T
T	F	F
F	T	T
F	F	T

If d and e are Boolean (0/1) variables:

- ▶ $d \rightarrow e \Leftrightarrow (\neg d) \vee e \Leftrightarrow (1 - d) + e \geq 1 \Leftrightarrow e \geq d$.
- ▶ Indeed, in case $d = 1$ we have $e \geq 1$ which means e must be true; in case $d = 0$ we want e to be unconstrained, and indeed we have $e \geq 0$ which is always true because lower-bound $lb(e) = 0$.

Linearisation of implications: Big-M

Consider $d \rightarrow x \geq 3$ with $0 \leq x \leq 10$.

- ▶ In case $d = 1$: we want $x \geq 3$ to be true. Let's rewrite the latter as $x - 3 \geq 0$.
So in case $d = 1$ we could write $x - 3 \geq (1 - d)$.

Linearisation of implications: Big-M

Consider $d \rightarrow x \geq 3$ with $0 \leq x \leq 10$.

- ▶ In case $d = 1$: we want $x \geq 3$ to be true. Let's rewrite the latter as $x - 3 \geq 0$. So in case $d = 1$ we could write $x - 3 \geq (1 - d)$.
- ▶ In case $d = 0$: we want $x - 3$ to be unconstrained. Is there a constant M such that $x - 3 \geq M$ would always be true? YES! its $M = lb(x - 3) = lb(x) - 3$.

Linearisation of implications: Big-M

Consider $d \rightarrow x \geq 3$ with $0 \leq x \leq 10$.

- ▶ In case $d = 1$: we want $x \geq 3$ to be true. Let's rewrite the latter as $x - 3 \geq 0$. So in case $d = 1$ we could write $x - 3 \geq (1 - d)$.
- ▶ In case $d = 0$: we want $x - 3$ to be unconstrained. Is there a constant M such that $x - 3 \geq M$ would always be true? YES! its $M = lb(x - 3) = lb(x) - 3$.
- ▶ So we can combine both cases in the linear inequality $x - 3 \geq M \cdot (1 - d)$ with $M = lb(x - 3)$ (or any value below that). It enforces the desired relation when $d = 1$ and it is unconstrained when $d = 0$.

Linearisation of implications: Big-M

Consider $d \rightarrow x \geq 3$ with $0 \leq x \leq 10$.

- ▶ In case $d = 1$: we want $x \geq 3$ to be true. Let's rewrite the latter as $x - 3 \geq 0$. So in case $d = 1$ we could write $x - 3 \geq (1 - d)$.
- ▶ In case $d = 0$: we want $x - 3$ to be unconstrained. Is there a constant M such that $x - 3 \geq M$ would always be true? YES! its $M = lb(x - 3) = lb(x) - 3$.
- ▶ So we can combine both cases in the linear inequality $x - 3 \geq M \cdot (1 - d)$ with $M = lb(x - 3)$ (or any value below that). It enforces the desired relation when $d = 1$ and it is unconstrained when $d = 0$.

The above technique is called a Big-M formulation of a logical constraint. It is used to model all sorts of implications and logical equivalences.

For example: $d \rightarrow (x = 3) \Leftrightarrow (d \rightarrow x \geq 3) \wedge (d \rightarrow x \leq 3)$
 $\Leftrightarrow (x - 3 \geq M_1 \cdot (1 - d)) \wedge (x - 3 \leq M_2 \cdot (1 - d)).$

Linearisation of implications: more Big-M

Consider $x \neq 3$. This is not easily rewritable in inequalities.

- ▶ It is logically equivalent to splitting into the disjunctive cases $(x > 3) \vee (x < 3)$.
- ▶ For integers: $(x \geq 4) \vee (x \leq 2)$, then $(d \vee e) \wedge (d \rightarrow x \geq 4) \wedge (e \rightarrow x \leq 2)$.
- ▶ Big-M reformulation: $x - 4 \geq M_1 \cdot (1 - d)$ and $x - 2 \leq M_2 \cdot (1 - e)$.
- ▶ Together: $(d + e \geq 1) \wedge (x - 4 \geq M_1 \cdot (1 - d)) \wedge (x - 2 \leq M_2 \cdot (1 - e))$

Linearisation of implications: more Big-M

Consider $x \neq 3$. This is not easily rewritable in inequalities.

- ▶ It is logically equivalent to splitting into the disjunctive cases $(x > 3) \vee (x < 3)$.
- ▶ For integers: $(x \geq 4) \vee (x \leq 2)$, then $(d \vee e) \wedge (d \rightarrow x \geq 4) \wedge (e \rightarrow x \leq 2)$.
- ▶ Big-M reformulation: $x - 4 \geq M_1 \cdot (1 - d)$ and $x - 2 \leq M_2 \cdot (1 - e)$.
- ▶ Together: $(d + e \geq 1) \wedge (x - 4 \geq M_1 \cdot (1 - d)) \wedge (x - 2 \leq M_2 \cdot (1 - e))$

But we can go one step further: we know that $(x \geq 4)$ and $(x \leq 2)$ can never happen at the same time, and hence neither can d and e . Combined with $d + e \geq 1$ we have $d + e = 1$ and hence $d = 1 - e$, allowing us to eliminate e :

- ▶ we **only need two constraints and one auxiliary variable** to linearize $x \neq 3$:
 $(x - 4 \geq M_1 \cdot (1 - d)) \wedge (x - 2 \leq M_2 \cdot d)$.

Linearisation of implications: more Big-M

Consider $x \neq 3$. This is not easily rewritable in inequalities.

- ▶ It is logically equivalent to splitting into the disjunctive cases $(x > 3) \vee (x < 3)$.
- ▶ For integers: $(x \geq 4) \vee (x \leq 2)$, then $(d \vee e) \wedge (d \rightarrow x \geq 4) \wedge (e \rightarrow x \leq 2)$.
- ▶ Big-M reformulation: $x - 4 \geq M_1 \cdot (1 - d)$ and $x - 2 \leq M_2 \cdot (1 - e)$.
- ▶ Together: $(d + e \geq 1) \wedge (x - 4 \geq M_1 \cdot (1 - d)) \wedge (x - 2 \leq M_2 \cdot (1 - e))$

But we can go one step further: we know that $(x \geq 4)$ and $(x \leq 2)$ can never happen at the same time, and hence neither can d and e . Combined with $d + e \geq 1$ we have $d + e = 1$ and hence $d = 1 - e$, allowing us to eliminate e :

- ▶ we **only need two constraints and one auxiliary variable** to linearize $x \neq 3$:
 $(x - 4 \geq M_1 \cdot (1 - d)) \wedge (x - 2 \leq M_2 \cdot d)$.

The same reasoning can be applied to $x \neq y$ and $\sum_j a_j \cdot x_j \neq b$.

Linearization of implications: general Big-M

For a general linear *indicator* constraint:

$$d = 1 \rightarrow \sum_j a_{ij}x_j \leq b \quad \Leftrightarrow \quad \sum_j a_{ij}x_j - b \leq M(1 - d)$$

We want a **small** $M = \max\{\sum_j a_{ij}x_j - b : \text{feasible } x\}$

Why?

- ▶ Big M's can lead to poor LP relaxations
- ▶ Really big M's can lead to numeric instability

Some ILP solvers accept indicator constraints, and for really big M's may defer their linearisation until it is smaller later in the branch-and-bound tree.

Linearization of implications: general Big-M

For a general linear *indicator* constraint:

$$d = 1 \rightarrow \sum_j a_{ij} x_j \leq b \quad \Leftrightarrow \quad \sum_j a_{ij} x_j - b \leq M(1 - d)$$

We want a **small** $M = \max\{\sum_j a_{ij} x_j - b : \text{feasible } x\}$

Why?

- ▶ Big M's can lead to poor LP relaxations
 - ▶ Really big M's can lead to numeric instability
- Some ILP solvers accept indicator constraints, and for really big M's may defer their linearisation until it is smaller later in the branch-and-bound tree.

Computing M by knowing what is 'feasible':

- ▶ The base case is the maximum over the box relaxation:
$$M = \sum_{j:a_{ij}<0} a_{ij} \cdot lb(x_j) + \sum_{j:a_{ij}>0} a_{ij} \cdot ub(x_j) - b$$
- ▶ More advanced approaches can use additional information, such as subsets of Boolean x that are at-most-1 (take their max instead of sum) or other constraints that lead to a smaller M .

Linearly encoding all values of a domain

Given an integer variable $1 \leq x \leq 10$.

To encode one of its values as $d_3 \leftrightarrow (x = 3)$ we would need 4 constraints: two for $d_3 \rightarrow (x = 3)$ and two for $(\neg d_3) \rightarrow x \neq 3$ (verify this yourself with the previous slide).

Linearly encoding all values of a domain

Given an integer variable $1 \leq x \leq 10$.

To encode one of its values as $d_3 \leftrightarrow (x = 3)$ we would need 4 constraints: two for $d_3 \rightarrow (x = 3)$ and two for $(\neg d_3) \rightarrow x \neq 3$ (verify this yourself with the previous slide).

However, if we know we are going to encode **each** value of x , e.g.

$d_1 \leftrightarrow (x = 1) \wedge \dots \wedge d_{10} \leftrightarrow (x = 10)$ then there is a much simpler encoding:

$$d_1 + d_2 + \dots + d_{10} = 1$$

$$1 \cdot d_1 + 2 \cdot d_2 + \dots + 10 \cdot d_{10} = x$$

So just 2 constraints to encode all 10 Booleans...

Linearly encoding all values of a domain

Given an integer variable $1 \leq x \leq 10$.

To encode one of its values as $d_3 \leftrightarrow (x = 3)$ we would need 4 constraints: two for $d_3 \rightarrow (x = 3)$ and two for $(\neg d_3) \rightarrow x \neq 3$ (verify this yourself with the previous slide).

However, if we know we are going to encode **each** value of x , e.g.

$d_1 \leftrightarrow (x = 1) \wedge \dots \wedge d_{10} \leftrightarrow (x = 10)$ then there is a much simpler encoding:

$$\begin{aligned}d_1 + d_2 + \dots + d_{10} &= 1 \\1 \cdot d_1 + 2 \cdot d_2 + \dots + 10 \cdot d_{10} &= x\end{aligned}$$

So just 2 constraints to encode all 10 Booleans...

You can go even further and *eliminate* all occurrences of x using the last equality...

Linearising ALLDIFFERENT()

Which of the following two ALLDIFFERENT($x_1, x_2, \dots, x_n$) decompositions do you think is best for an ILP solver?

$$x_i \neq x_j \quad \text{for } i = 1, \dots, n \quad \text{for } j = i + 1, \dots, n$$

or:

$$\sum_{v \in D(x_i)} v \cdot d_{iv} = x_i \quad \text{for } i = 1, \dots, n$$

$$\sum_{v \in D(x_i)} d_{iv} = 1 \quad \text{for } i = 1, \dots, n$$

$$\sum_{i=1..n} d_{iv} \leq 1 \quad \text{for } v = \min_i(D(x_i)), \dots, \max_i(D(x_i))$$

Linearising ALLDIFFERENT()

Which of the following two ALLDIFFERENT($x_1, x_2, \dots, x_n$) decompositions do you think is best for an ILP solver?

$$x_i \neq x_j \quad \text{for } i = 1, \dots, n \quad \text{for } j = i + 1, \dots, n$$

or:

$$\sum_{v \in D(x_i)} v \cdot d_{iv} = x_i \quad \text{for } i = 1, \dots, n$$

$$\sum_{v \in D(x_i)} d_{iv} = 1 \quad \text{for } i = 1, \dots, n$$

$$\sum_{i=1..n} d_{iv} \leq 1 \quad \text{for } v = \min_i(D(x_i)), \dots, \max_i(D(x_i))$$

The latter is the **linear (flow-graph) decomposition** from lecture 3. Creating the 'domain value' d_{iv} auxiliary variables per integer variable allows for a tighter LP relaxation than when creating Big-M-ed 'not equals' auxiliary variables per pair of integer variables.

Linearising global constraints

Creating 'domain value' d_{iv} auxiliary variables per integer variable x_i is a good **pattern** for many global constraints that do domain-consistent reasoning!

Other examples, let $X = [x_1, x_2, \dots, x_n]$ an array of integer decision variables:

- ▶ $\text{COUNT}(\text{EQ}(X, v, \text{res}))$: $(\text{res} = \sum_i d_{iv})$
- ▶ $\text{GLOBALCARDINALITYCOUNT}(X, Vs, Res)$: $\forall v \in Vs : (Res_v = \sum_i d_{iv})$
- ▶ $\text{NVALUEEQ}(X, \text{res})$: $(\text{res} = \sum_v b_v) \wedge (\forall v \in Vals : (b_v = (\sum_i d_{iv} \geq 1)))$
- ▶ $\text{ELEMENTEQ}(Arr, x_i, \text{res})$: $(\text{res} = \sum_v Arr_v \cdot d_{iv})$
- ▶ Also for $\text{TABLE}(X, T)$, $\text{MAXEQ}(X, \text{res})$, $\text{PROD}(x_i, x_j, \text{res})$ and more.

Linearising global constraints

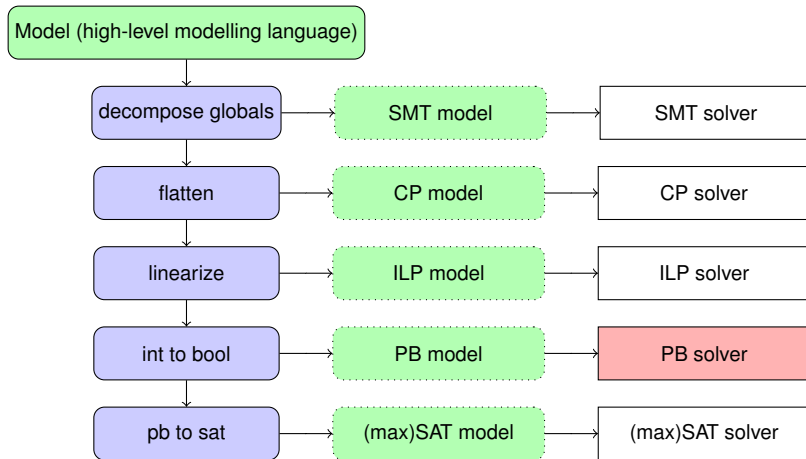
Creating 'domain value' d_{iv} auxiliary variables per integer variable x_i is a good **pattern** for many global constraints that do domain-consistent reasoning!

Other examples, let $X = [x_1, x_2, \dots, x_n]$ an array of integer decision variables:

- ▶ $\text{COUNT}(X, v, res): (res = \sum_i d_{iv})$
- ▶ $\text{GLOBALCARDINALITYCOUNT}(X, Vs, Res): \forall v \in Vs : (Res_v = \sum_i d_{iv})$
- ▶ $\text{NVALUEEQ}(X, res): (res = \sum_v b_v) \wedge (\forall v \in Vals : (b_v = (\sum_i d_{iv} \geq 1)))$
- ▶ $\text{ELEMENTEQ}(Arr, x_i, res): (res = \sum_v Arr_v \cdot d_{iv})$
- ▶ Also for $\text{TABLE}(X, T)$, $\text{MAXEQ}(X, res)$, $\text{PROD}(x_i, x_j, res)$ and more.

In fact, ILP modellers typically only use integer variables for *relaxable* quantities (where a fractional $x_i = 3.6$ makes sense, like quantity, volume, flow), and otherwise directly model over Boolean variables for the discrete options d_{iv} .

Overview



Pseudo-Boolean (PB) Solving

Only accepts **Pseudo-Boolean Constraints**: $\sum_{i=1}^n a_i x_i \leq b$, where a_i, b are integer constants and x_i are Boolean decision variables.

Extends SAT by allowing linear inequalities over Boolean variables (equivalently: ILP restricted to 0/1 variables).

Pseudo-Boolean (PB) Solving

Only accepts **Pseudo-Boolean Constraints**: $\sum_{i=1}^n a_i x_i \leq b$, where a_i, b are integer constants and x_i are Boolean decision variables.

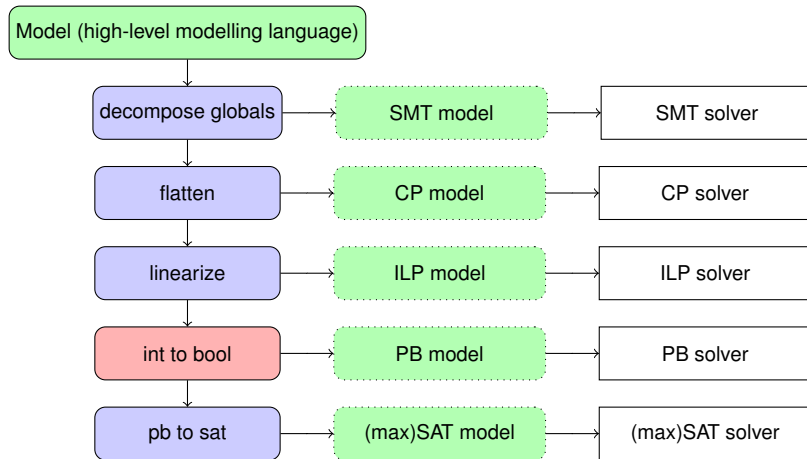
Extends SAT by allowing linear inequalities over Boolean variables (equivalently: ILP restricted to 0/1 variables).

Two possible solving techniques:

- ▶ Eagerly encode into SAT (see next part)
- ▶ Develop a SAT-like solver with pseudo-boolean propagation and pseudo-boolean conflict analysis (not covered in lecture)
Example such solvers: RoundingSAT, Exact

Pseudo-Boolean **Optimisation** (PBO): optimising a linear objective over Boolean variables.

Overview



Encoding into (pseudo)Boolean constraints

Challenges:

- ▶ How to encode an integer variable into a collection of Boolean variables?
- ▶ How to encode a constraint on integer variables into (a collection of) constraints on Boolean variables?

Considerations:

- ▶ We want few variables.
- ▶ We want few constraints, or short constraints.
- ▶ We want strong propagation.

As usual, there are many possibilities, which make different trade-offs.

Encoding an Integer Variable

Well-known encodings, described on the next slides:

- ▶ **Direct (or value or sparse or one-hot) encoding:**
a Boolean 'equality' variable for each value in the domain.
- ▶ **Order encoding:**
a Boolean 'inequality' variable for each value in the domain.
- ▶ **Log (or bit or binary) encoding:**
a Boolean variable for each bit in the base-2 representation of the largest domain value.

Encodings: example with overview

Encodings for $x \in \{0, 1, 2, 3\}$:

x	Direct encoding				Order encoding			Log encoding	
	$b_{[x=0]}$	$b_{[x=1]}$	$b_{[x=2]}$	$b_{[x=3]}$	$b_{[x \geq 1]}$	$b_{[x \geq 2]}$	$b_{[x \geq 3]}$	$b_{\text{bit}^1(x)}$	$b_{\text{bit}^0(x)}$
0	1	0	0	0	0	0	0	0	0
1	0	1	0	0	1	0	0	0	1
2	0	0	1	0	1	1	0	1	0
3	0	0	0	1	1	1	1	1	1

Direct Encoding of an Integer Variable

Consider an integer variable x with domain $0..n$:

- ▶ Create a Boolean variable $b_{[x=k]}$ for all $k \in \{0..n\}$
- ▶ The variable $b_{[x=k]}$ is **true** if and only if $x = k$ holds
- ▶ Consistency constraint:
 - ▶ Variable x has exactly one value from domain: $\sum_{k=0}^n b_{[x=k]} = 1$
- ▶ Example encodings of simple constraints:
 - ▶ The constraint $x \neq k$ is encoded as $\neg b_{[x=k]}$
 - ▶ The constraint $x < k$ is encoded as $\bigwedge_{j \in \{k..n\}} \neg b_{[x=j]}$
 - ▶ In any constraint, can replace x by $\sum_k k \cdot b_{[x=k]}$

Order Encoding of an Integer Variable

Consider an integer variable x with domain $0..n$:

- ▶ Create a Boolean variable $b_{[x \geq k]}$ for all $k \in \{1..n\}$.
- ▶ The variable $b_{[x \geq k]}$ is **true** if and only if $x \geq k$ holds.
- ▶ Consistency constraint:
 - ▶ Monotone ordering:
$$\bigwedge_{k \in 1..(n-1)} (b_{[x \geq k+1]} \rightarrow b_{[x \geq k]})$$
- ▶ Example encodings of simple constraints
(assume $b_{[x \geq 0]}$ returns True, $b_{[x \geq n+1]}$ returns False):
 - ▶ The constraint $x = k$ is encoded as $b_{[x \geq k]} \wedge \neg b_{[x \geq k+1]}$
 - ▶ The constraint $x \neq k$ is encoded as $\neg b_{[x \geq k]} \vee b_{[x \geq k+1]}$
 - ▶ In any constraint, can replace x by $\sum_k b_{[x \geq k]}$

Log Encoding of an Integer Variable

Consider an integer variable x with domain $0..n$ (so $n + 1$ values):

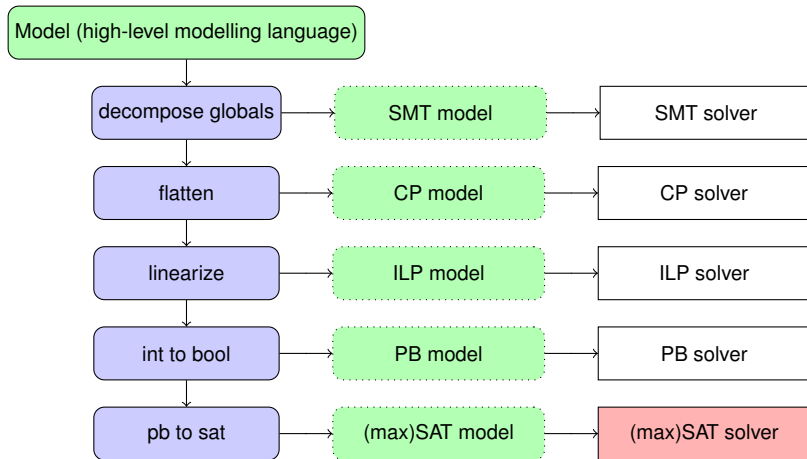
- ▶ Encode the value of x in binary, using just $\lceil \log_2(n + 1) \rceil$ Boolean variables.
- ▶ Let b_i a Boolean variable representing the i -th bit in the binary encoding of x (counting from the right, least significant bit).
- ▶ In any constraint, can replace x by:

$$x = \sum_{i=0}^{\lceil \log_2(n+1) \rceil - 1} b_i \cdot 2^i$$

where $b_i \in \{0, 1\}$.

- ▶ Consistency constraint, in case $n + 1$ is not a power of 2:
 - ▶ Upper bound of domain: $\sum_{i=0}^{\lceil \log_2(n+1) \rceil - 1} b_i \cdot 2^i \leq n$

Overview



The SAT Problem

- ▶ **Decision variable** x : takes value **true** ($= 1$) or **false** ($= 0$)
- ▶ **Literal** ℓ : variable x or its negation $\neg x$
(which we will from now on write as $\bar{x}$ instead of $\neg x$)
- ▶ **Primitive constraint** a **clause** $C = \ell_1 \vee \dots \vee \ell_k$ disjunction of literals
(considered as sets, so no repetitions and order irrelevant)
- ▶ **Conjunctive normal form** (CNF) formula $F = C_1 \wedge \dots \wedge C_m$ conjunction of clauses

The SAT Problem

Given a CNF formula F , is it satisfiable?

For instance, what about:

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge \\ (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

► $p \vee \bar{u}$ propagates $u \mapsto 0$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

- ▶ $p \vee \bar{u}$ propagates $u \mapsto 0$
- ▶ $q \vee r$ propagates $r \mapsto 1$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

- ▶ $p \vee \bar{u}$ propagates $u \mapsto 0$
- ▶ $q \vee r$ propagates $r \mapsto 1$
- ▶ Then $\bar{r} \vee w$ propagates $w \mapsto 1$

SAT propagation: unit propagation

Unit Propagation

Clause C **unit propagates** ℓ under partial assignment ρ if ρ falsifies all literals in C except ℓ .

Example: Unit propagate for $\rho = \{p \mapsto 0, q \mapsto 0\}$ on

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

- ▶ $p \vee \bar{u}$ propagates $u \mapsto 0$
- ▶ $q \vee r$ propagates $r \mapsto 1$
- ▶ Then $\bar{r} \vee w$ propagates $w \mapsto 1$
- ▶ No further unit propagations

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

$$r \stackrel{q \vee r}{=} 1$$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$p \stackrel{d}{=} 0$
$u \stackrel{p \vee \bar{u}}{=} 0$
$q \stackrel{d}{=} 0$
$r \stackrel{q \vee r}{=} 1$
$w \stackrel{\bar{r} \vee w}{=} 1$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

$$r \stackrel{q \vee r}{=} 1$$

$$w \stackrel{\bar{r} \vee w}{=} 1$$

$$x \stackrel{d}{=} 0$$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$p \stackrel{d}{=} 0$
$u \stackrel{p \vee \bar{u}}{=} 0$
$q \stackrel{d}{=} 0$
$r \stackrel{q \vee r}{=} 1$
$w \stackrel{\bar{r} \vee w}{=} 1$
$x \stackrel{d}{=} 0$
$y \stackrel{u \vee x \vee y}{=} 1$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$p \stackrel{d}{=} 0$
$u \stackrel{p \vee \bar{u}}{=} 0$
$q \stackrel{d}{=} 0$
$r \stackrel{q \vee r}{=} 1$
$w \stackrel{\bar{r} \vee w}{=} 1$
$x \stackrel{d}{=} 0$
$y \stackrel{u \vee x \vee y}{=} 1$
$z \stackrel{x \vee \bar{y} \vee z}{=} 1$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

Add to assignment *trail*

Continue until satisfying assignment or **conflict**

Conflict-Driven Clause Learning (CDCL)

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$p \stackrel{d}{=} 0$
$u \stackrel{p \vee \bar{u}}{=} 0$
$q \stackrel{d}{=} 0$
$r \stackrel{q \vee r}{=} 1$
$w \stackrel{\bar{r} \vee w}{=} 1$
$x \stackrel{d}{=} 0$
$y \stackrel{u \vee x \vee y}{=} 1$
$z \stackrel{x \vee \bar{y} \vee z}{=} 1$
$\bar{y} \vee \bar{z}$ $\perp$

Search decision

Free choice to assign value to variable

Notation $p \stackrel{d}{=} 0$

Unit propagation

Forced choice to avoid falsifying clause

Given $p = 0$, clause $p \vee \bar{u}$ forces $u = 0$

Notation $u \stackrel{p \vee \bar{u}}{=} 0$ ($p \vee \bar{u}$ is reason clause)

Always propagate if possible, otherwise decide

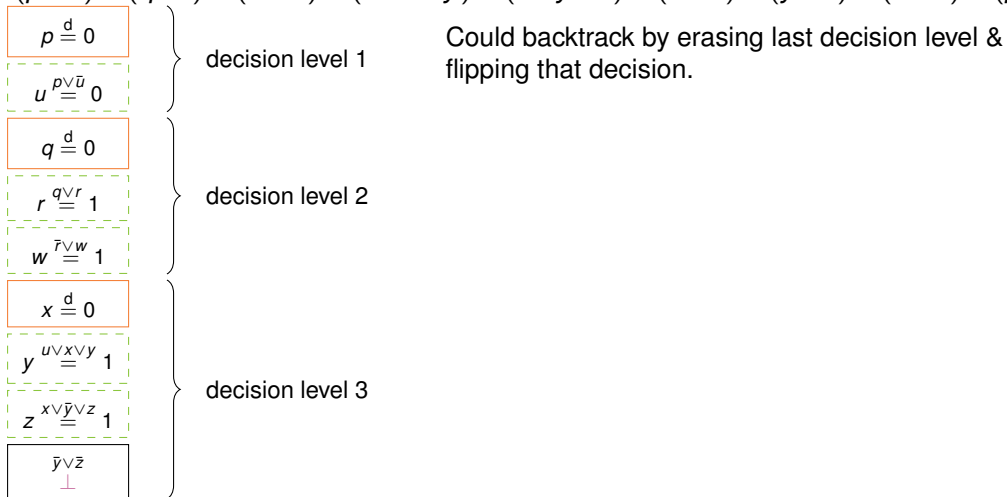
Add to assignment *trail*

Continue until satisfying assignment or **conflict**

CDCL, conflict analysis

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$



CDCL, conflict analysis

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

$$r \stackrel{q \vee r}{=} 1$$

$$w \stackrel{\bar{r} \vee w}{=} 1$$

$$x \stackrel{d}{=} 0$$

$$y \stackrel{u \vee x \vee y}{=} 1$$

$$z \stackrel{x \vee \bar{y} \vee z}{=} 1$$

$$\bar{y} \vee \bar{z} \\ \perp$$

decision level 1

decision level 2

decision level 3

Could backtrack by erasing last decision level & flipping that decision.

But want to **learn** from conflict and cut away as much of search space as possible.

CDCL, conflict analysis

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

$$r \stackrel{q \vee r}{=} 1$$

$$w \stackrel{\bar{r} \vee w}{=} 1$$

$$x \stackrel{d}{=} 0$$

$$y \stackrel{u \vee x \vee y}{=} 1$$

$$z \stackrel{x \vee \bar{y} \vee z}{=} 1$$

$$\bar{y} \vee \bar{z} \\ \perp$$

$$x \vee \bar{y}$$

Could backtrack by erasing last decision level & flipping that decision.

But want to **learn** from conflict and cut away as much of search space as possible.

Case analysis over z for last two clauses:

- ▶ $x \vee \bar{y} \vee z$ wants $z = 1$
- ▶ $\bar{y} \vee \bar{z}$ wants $z = 0$
- ▶ Resolve clauses by merging them & removing z
— must satisfy $x \vee \bar{y}$

CDCL, conflict analysis

CDCL combines **unit propagation** and **search** with **conflict analysis**.

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

$$p \stackrel{d}{=} 0$$

$$u \stackrel{p \vee \bar{u}}{=} 0$$

$$q \stackrel{d}{=} 0$$

$$r \stackrel{q \vee r}{=} 1$$

$$w \stackrel{\bar{r} \vee w}{=} 1$$

$$x \stackrel{d}{=} 0$$

$$y \stackrel{u \vee x \vee y}{=} 1$$

$$z \stackrel{x \vee \bar{y} \vee z}{=} 1$$

$$\bar{y} \vee \bar{z} \perp$$

$$u \vee x$$

$$x \vee \bar{y}$$

Could backtrack by erasing last decision level & flipping that decision.

But want to **learn** from conflict and cut away as much of search space as possible.

Case analysis over z for last two clauses:

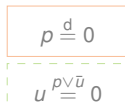
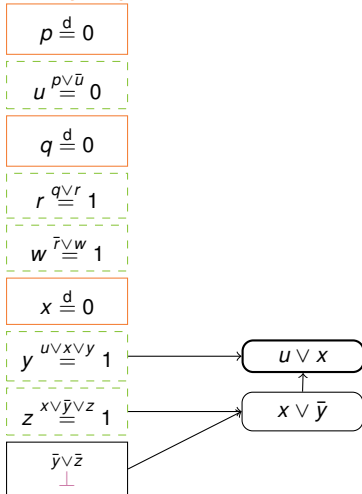
- ▶ $x \vee \bar{y} \vee z$ wants $z = 1$
- ▶ $\bar{y} \vee \bar{z}$ wants $z = 0$
- ▶ Resolve clauses by merging them & removing z — must satisfy $x \vee \bar{y}$

Repeat until *UIP clause* with only 1 variable at conflict level after last decision — **learn** it.

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates

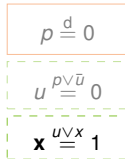
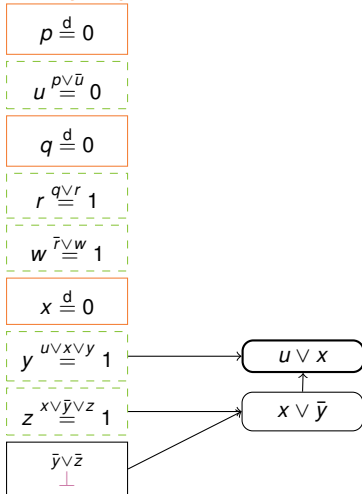


Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



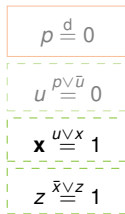
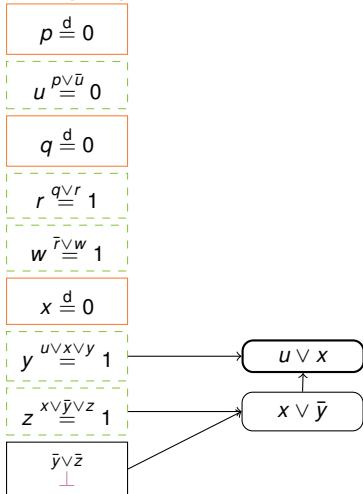
Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

Now UIP literal (**x**) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

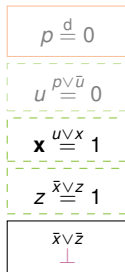
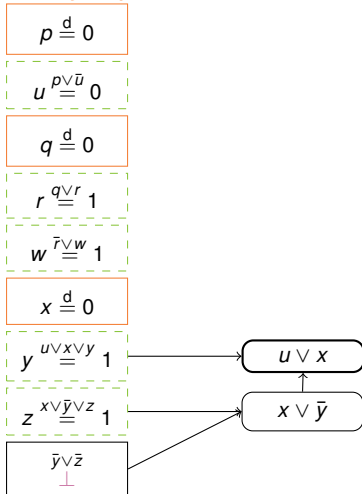
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

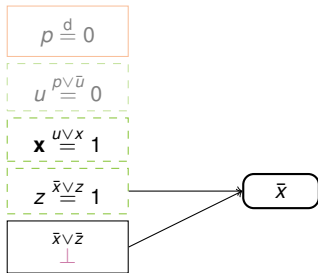
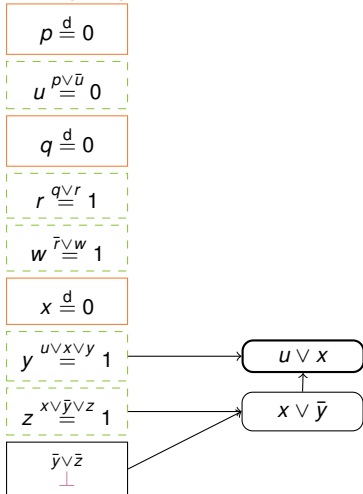
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

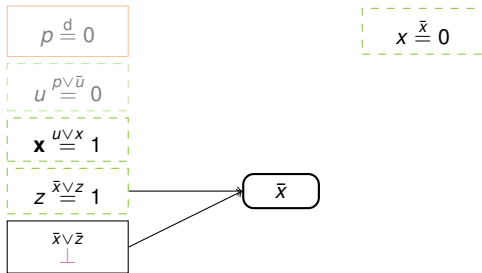
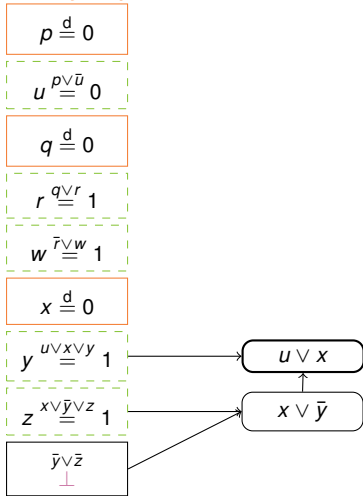
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

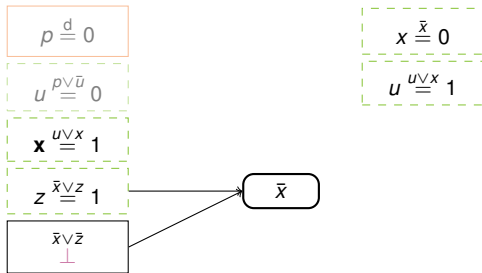
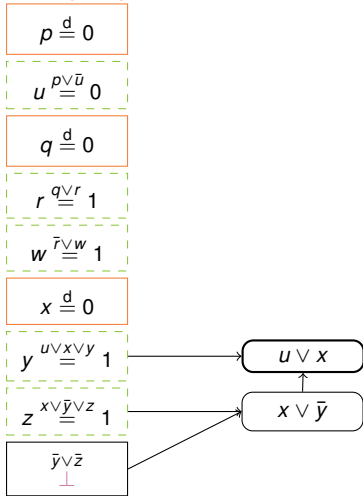
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

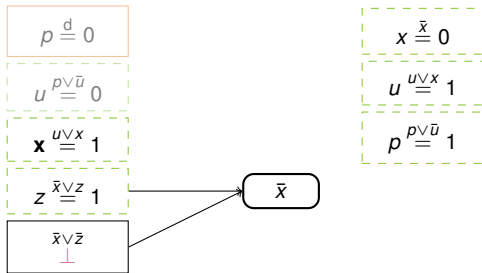
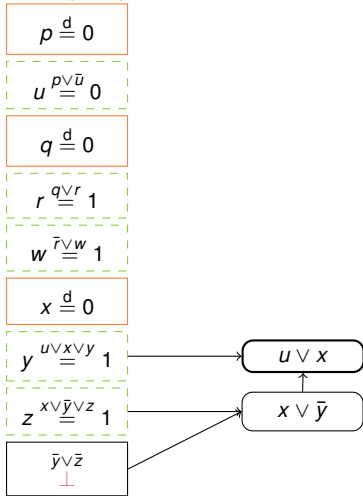
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

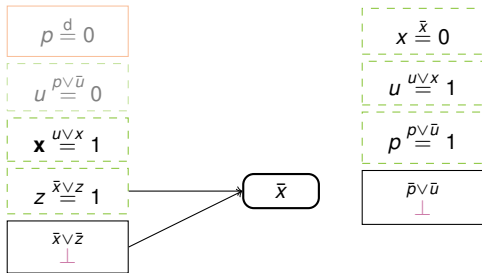
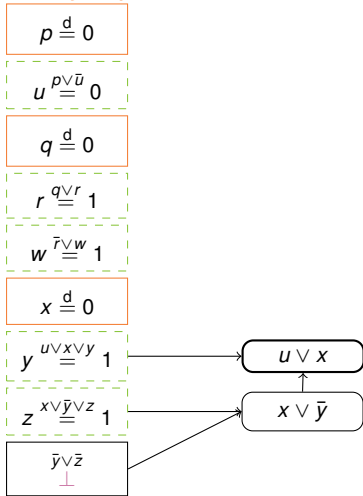
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

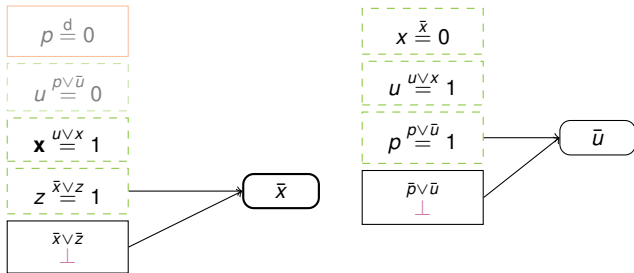
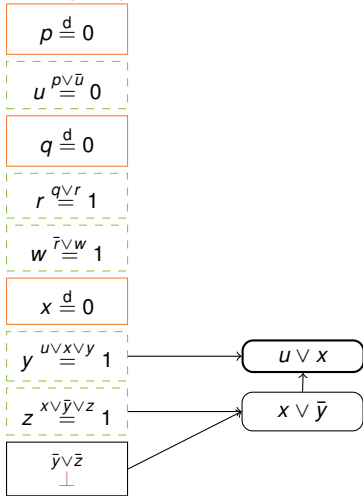
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

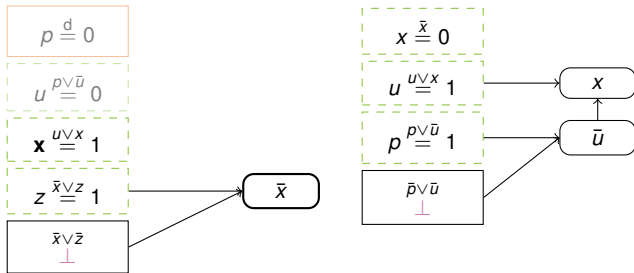
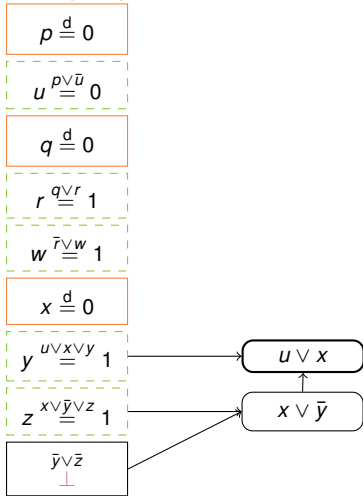
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Assertion level 1: 2nd largest level in learned clause, by u . Trim trail to that level.

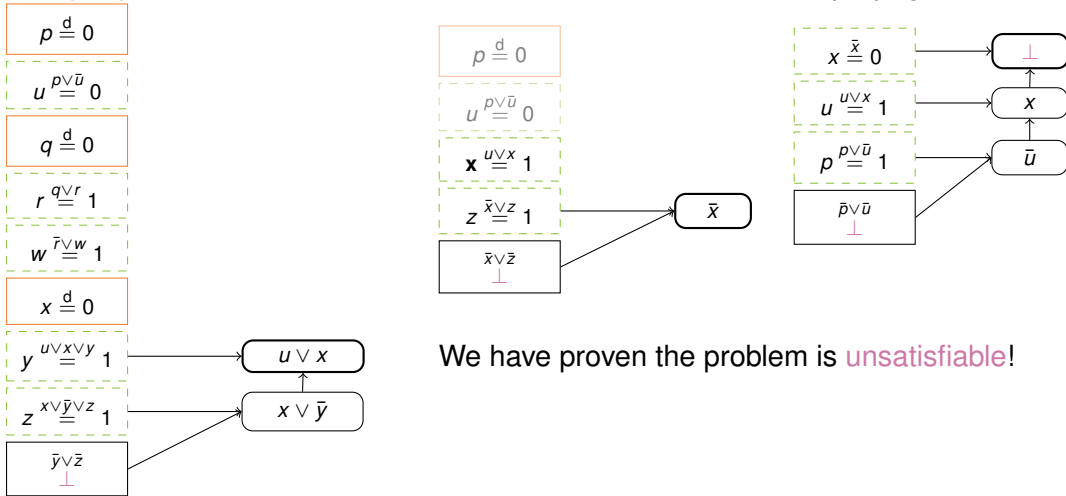
Now UIP literal (x) guaranteed to flip by learned clause. Note that this is a **propagation**, not a **decision**.

Then continue...

CDCL, backjumping

$$(p \vee \bar{u}) \wedge (q \vee r) \wedge (\bar{r} \vee w) \wedge (u \vee x \vee y) \wedge (x \vee \bar{y} \vee z) \wedge (\bar{x} \vee z) \wedge (\bar{y} \vee \bar{z}) \wedge (\bar{x} \vee \bar{z}) \wedge (\bar{p} \vee \bar{u})$$

Backjump: undo max nr. of decision levels while learned clause propagates



Efficient SAT solvers

Many additional techniques:

- ▶ 2-watched literals: a clause can only unit-propagate if it has only 1 unassigned literal remaining, so watching 2 unassigned literals is enough (if you 'pass on' the watch on assignment)
- ▶ variable ordering heuristics, e.g. VSIDS/VMTF (activity/conflict-like)
- ▶ phase saving: remember previous assignments to guide future decisions
- ▶ restarts: occasionally restart from decision-level 0
- ▶ pre-processing and in-processing: simplify formula during search. E.g. variable elimination (a generalisation of pure literal detection), blocked clause elimination, equivalence detection, ...
- ▶ clause database reduction: periodically remove less useful learned clauses (to manage memory use and memory compactness)
- ▶ fast data structures

MaxSAT Solvers

Adds a linear objective to be maximized.

Three main categories:

- ▶ Linear SAT-UNSAT search (*solution-improving search*)
 1. Call SAT solver to find some solution
 2. Add clauses encoding “I want a better solution”
 3. Repeat (last found solution is optimal)

MaxSAT Solvers

Adds a linear objective to be maximized.

Three main categories:

- ▶ Linear SAT-UNSAT search (*solution-improving search*)
 1. Call SAT solver to find some solution
 2. Add clauses encoding “I want a better solution”
 3. Repeat (last found solution is optimal)
- ▶ Core-guided search
 1. Call SAT solver to find solution under most optimistic assumptions
 2. If impossible, rewrite objective given output of SAT solver
 3. Repeat (first solution is optimal)

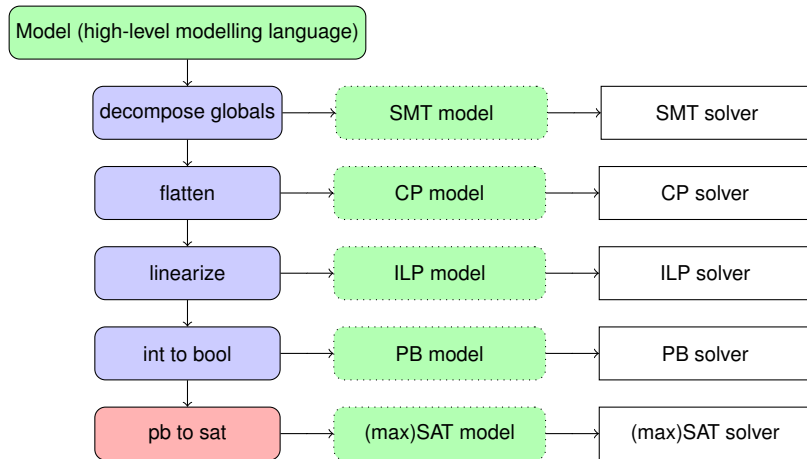
MaxSAT Solvers

Adds a linear objective to be maximized.

Three main categories:

- ▶ Linear SAT-UNSAT search (*solution-improving search*)
 1. Call SAT solver to find some solution
 2. Add clauses encoding “I want a better solution”
 3. Repeat (last found solution is optimal)
- ▶ Core-guided search
 1. Call SAT solver to find solution under most optimistic assumptions
 2. If impossible, rewrite objective given output of SAT solver
 3. Repeat (first solution is optimal)
- ▶ Implicit Hitting Set
 1. Call SAT solver to find solution under most optimistic assumptions
 2. Use hitting set solver (ILP solver) to recompute what most possible optimistic assumptions are
 3. Repeat (first solution is optimal)

Overview



Encoding PB to SAT

Example: $\sum_{k \in \{1..n\}} b_k = 1$ (as used e.g. in the direct encoding of integers)

$$\begin{aligned}\sum_{k \in \{1..n\}} b_k = 1 &\Leftrightarrow \sum_{k \in \{1..n\}} b_k \geq 1 \wedge \sum_{k \in \{1..n\}} b_k \leq 1 \\ \sum_{k \in \{1..n\}} b_k \geq 1 &\Leftrightarrow \bigvee_{k \in 1..n} b_k \\ \sum_{k \in \{1..n\}} b_k \leq 1 &\Leftrightarrow \bigwedge_{i,j \in 1..n, i < j} (b_i + b_j \leq 1) \\ &\Leftrightarrow \bigwedge_{i,j \in 1..n, i < j} (\neg b_i \vee \neg b_j)\end{aligned}$$

From 1 PB constraint over m variables,
to 1 clause over m variables + $m * (m - 1)/2$ binary clauses.

Encoding any PB Constraint to SAT

Pseudo-Boolean Constraints: $\sum_{i=1}^n a_i x_i \leq b$, with a_i, b integers, x_i Boolean variables. Many encoding techniques exist, e.g.: (details beyond the scope of this course)

- ▶ **Adder Networks:**

- ▶ Use a network of binary adder circuits (like in hardware) to encode the sum.
- ▶ Network is polynomial in size, but has weaker propagation (not *arc consistent*).

- ▶ **Sorting Networks:**

- ▶ Use a network of comparators to sort inputs, enforcing PB constraints on sums.
- ▶ Strong propagation & works well for small/medium n ; polynomial size, but can be large.

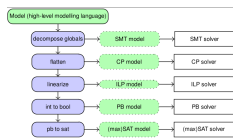
- ▶ **Totalizer Encoding:**

- ▶ Introduces auxiliary variables for cumulative sums.
- ▶ Useful for incremental solving; strong propagation, but size can grow quickly (still polynomial).

- ▶ **Binary Decision Diagrams (BDD):**

- ▶ Use a BDD over the x_i s to compactly represent the allowed values.
- ▶ Strong propagation, but building BDDs can be computationally expensive &

Choosing between solver technologies...



Better understand what a solver CAN do, and what it is GOOD at (or bad).

- ▶ SAT: recommended if very Boolean
- ▶ PB: recommended if very Boolean but also many linear sums
- ▶ ILP: recommended if many linear constraints and optimizing the 'continuous relaxation' is meaningful
- ▶ SMT: recommended if many disjunctive constraints (SAT reasoning + LIA reasoning)
- ▶ CP: recommended if you have many global constraints

(but there is no rulebook)

Some Caveats

- ▶ Each problem can be modelled in many different ways.
- ▶ Different models of the same problem are better suited for different backends.
- ▶ Performance on small instances does not always scale to larger instances.
- ▶ Sometimes, a good **search strategy** is more important than a good model.
- ▶ Not all backends of the same technology have comparable performance.
- ▶ Some pure problems can be solved by specialist solvers, such as **Concorde** for the travelling salesperson problem, but real-life side constraints often make them inapplicable. space.

Take-Home Message:

- ▶ There are many solving technologies and ways to encode to them.
- ▶ It is useful to understand the commonalities and differences.
- ▶ No solving technology can be universally better than all the others, unless $P = NP$.

👉 With solver-independent frameworks: can try multiple ones!